

Governed Agent Morphogenesis: Runtime Organizational Reconfiguration with Bounded Authority and Causal Continuity

Research preprint v0.2 - 2026-09-19

Douglas Rodríguez

Reference implementation: AgentPlat Agent Morphogenesis.

Status: systems protocol with a reproducible local boundary pilot.

Abstract

Agent collectives may need to recruit specialists, create agents, transfer work, and retire participants while a mission is in progress. Each change can cross independently authoritative lifecycle, membership, work, and action services. We present governed agent morphogenesis: a transition protocol that binds an organizational proposal to an exact decision, bounded resources, owner-specific effects, and a reconstructible successor history. Persistent Agent Rooms provide one integration context in which work artifacts and human decisions remain addressable as participants change. We give conditional safety arguments, map their obligations to AgentPlat's reference implementation, and release an eleven-case local boundary pilot. Under a prescribed lost-response schedule with process termination, both the reference runtime and a durable-pattern comparator recover one synthetic effect; a fresh-identity retry ablation produces two. A separate boundary probe accepts one morphology successor while retaining an effect from the losing proposal, demonstrating why head consistency is weaker than transition atomicity. These observations support bounded recovery behavior and expose integration obligations. They do not establish organizational improvement or deployment reliability. The protocol makes organizational change inspectable without transferring effect authority to the coordinator.

Keywords: multi-agent systems; organizational adaptation; agent governance; durable execution; causal continuity; runtime reconfiguration.

1. Introduction

Consider an Agent Room in which a person and an agent collective prepare a technical proposal. Its initial members can research requirements and draft an answer, but a new requirement needs a specialist. The collective may recruit an existing agent or instantiate a temporary one, assign bounded work, incorporate its output, and eventually remove it. Even this small adaptation crosses several boundaries: a factory materializes an agent, membership admits an identity, a team incorporates a participant, work contracts allocate responsibility, and an action gateway determines which external effects are permitted.

Changing a list of agents does not coordinate those boundaries. A decision may expire before it is used. Two controllers may propose different successors from the same organization. A provider may perform an effect and lose its acknowledgement. Retirement may remove a resource while leaving actionable work behind. A resumed controller may encounter evidence that an operation was prepared without knowing whether it completed.

This paper asks: **under which explicit assumptions can an agent collective transform its operational organization while preserving bounded authority and a reconstructible account of ongoing work?** We use *morphogenesis* to denote mission-driven changes in operational form. The term is an engineering abstraction; it does not imply biological development, autonomous evolution of model weights, or an intrinsic tendency toward better organizations.

Organizational modeling, architectural adaptation, and compensation already provide important foundations. AgentPlat's proposed contribution is a concrete composition: each organizational transition connects observation, intent, authorization, owner-specific effects, recovery, and a versioned morphology head. The morphology controller coordinates these owners without replacing their authority.

The paper makes three contributions: (1) a transition model separating organizational intent, decision authority, and effect ownership; (2) a recoverable protocol with explicit assumptions and implementation obligations for

successor selection, retirement, and evidence continuity; and (3) an open-source reference mapping and reproducible boundary pilot, including a durable-pattern comparator and a counterexample to interpreting head consistency as atomic reconfiguration. Protocol safety and organizational utility are evaluated as separate questions.

The scope is deliberately bounded. The core protocol concerns recruitment, catalog-based instantiation, detachment, and retirement, with selected advanced operators illustrating composition. Strategy learning, profile synthesis, whole-collective evolution, and constitutional amendment are extensions rather than prerequisites for the claims made here. No new model training algorithm or universal organizational optimizer is proposed.

2. Related work and positioning

2.1 Explicit agent organizations

Agent organizations have long been modeled through roles, groups, missions, permissions, and obligations. The *Moise* project explicitly represents organization for agent reasoning and infrastructure enforcement. *Dignum* and *Dignum* relate organizational structure, objectives, and agent actions in a logical framework. These precedents mean that explicit organization and normative constraints are established research topics, not novelty claims of this paper. [Moise project](#); [Dignum and Dignum, 2012](#).

Runtime reorganization itself is also established. Sorici et al. describe a *JaCaMo ReorgBoard*, an implementation plan, and organizational rules for coordinating reorganization. Their plan separates stopping an organizational entity, changing its specification, and starting its successor [10, Sections 3-4]. *ParaMoise* represents reorganization through workflows and read/write locks over the affected parts of an organization; compatible reorganizations can proceed concurrently [11, Section 3.3]. These are direct predecessors, not merely background on static roles. [JaCaMo reorganization](#); [ParaMoise](#).

Our focus is a particular cross-service contract: bind an exact organizational decision to owner-admitted effects, preserve their identities across uncertain outcomes, and record accepted lineage without treating that lineage as a global transaction. The comparison below identifies mechanisms described in the consulted sources. It does not claim their implementations lack unreported controls.

Precedent	Mechanism established in the source	Boundary addressed here
JaCaMo reorganization [10]	Norm-regulated reorganization and coordinated implementation plans	Stable effect identity and ambiguous outcomes across independently authoritative services
ParaMoise [11]	Workflow-based reorganization and scoped read/write exclusion	Receipt-linked successor commitment when owner effects are not globally atomic
Sagas [8]	Compensating partially completed long-lived transactions	Organizational admission, effect authority and post-commit lineage
MorphAgent [5]	Feedback-driven profile adaptation	How a selected change is authorized and recovered

Agent Rooms [12] supplies a complementary abstraction: a persistent collaboration container for human and agent work. *Morphogenesis* specifies transitions of the organization operating in that context. We do not claim to originate organizational adaptation, normative governance, durable execution, or artifact-oriented collaboration.

2.2 Adaptation in LLM-based collectives

AgentVerse dynamically adjusts group composition through its collaboration process. *MorphAgent* evolves profiles and roles using task feedback. *G-Designer* constructs task-conditioned communication topologies. These works motivate adaptive collectives and provide relevant comparisons for future utility experiments. [AgentVerse](#); [MorphAgent v2](#); [G-Designer](#).

The detailed profile-update mechanism in MorphAgent v2 (Section 3) informs the comparison above; a profile proposal does not itself specify how heterogeneous effect owners recover an interrupted change. This is a scope distinction, not evidence that another implementation lacks recovery controls. Adaptation algorithms could supply candidate organizations to Morphogenesis, provided their proposals remain subject to the same admission rules. The initial causal evaluation therefore compares controlled implementations of the same adaptation policy; cross-framework benchmarking is a later experiment with additional integration confounders.

2.3 Architectural adaptation and durable effects

Rainbow uses architectural models and external adaptation mechanisms to support self-adaptive software. Morphogenesis similarly makes the managed structure explicit, while specializing the transition boundary to agent membership, work, and action authority. The adaptation-control pattern is inherited; the proposed contribution lies in how its agent-specific responsibilities compose. [Garlan et al., 2004](#).

Sagas established compensation as a way to address partially executed long-lived transactions. Morphogenesis reuses this idea for declared reversible effects. It does not treat compensation as an atomic reversal of arbitrary external actions. Stable operation identities, durable journals, and compare-and-swap are engineering mechanisms used in the composition, not inventions claimed here. [Garcia-Molina and Salem, 1987](#).

Finally, causal ordering in distributed systems differs from a globally simultaneous observation. Our model records dependencies among observations, decisions, and receipts; it does not reconstruct a universal snapshot or establish that all relevant events have been observed. [Lamport, 1978](#).

3. System model and assumptions

3.1 Organization and observation

Let a mission scope be $s=(\text{tenant}, \text{mission})$. For analysis, represent a morphology projection at epoch e as

$$M_e=(s,e,V_e,R_e,T_e,W_e,H_e,P_e).$$

Here V_e contains references to participants, R_e to roles and profiles, T_e to team or topology structure, W_e to work and continuity records, H_e to authenticated source heads, and P_e to policy. This tuple is paper notation, not an additional runtime schema. Each component is a bounded projection or reference to state owned elsewhere.

A source head binds its source identity, revision, digest, scope, authentication evidence, observation time, and expiry. Policy determines the sources required for a particular operator. A missing or stale required source prevents admission. Failure to discover an eligible candidate means absence from this bounded view, not absence from the entire network.

The narrow authoritative morphology head is

$$h_e=(s,e,\text{parentDigest},\text{snapshotDigest},\text{receiptDigest},\text{revision}).$$

It records an accepted organizational successor. It is not the membership registry, the work scheduler, or an authorization database. Subsystem epochs may advance independently; use-time owner checks remain necessary after observation and decision.

3.2 Proposals, decisions, and effects

A proposal p binds the current morphology epoch and digest, a target projection, an operator plan, policy, budget envelope, expiry, and process definition. Its operator plan is a dependency graph $G_p=(N_p,E_p)$. Each effectful node names its owning subsystem, stable operation identity, prerequisites, expected receipt, and any declared compensation.

A decision d approves the exact candidate under a configured route: authorized agent, authorized person, local policy, collective decision, or a composite. Agent membership or proposal authorship alone supplies no decision mandate. Human review is selected by policy rather than required by definition.

We distinguish transition admission from effect admission:

$\text{Admit}(p,d,h,t) = \text{BindingMatch}(p,d,h) \wedge \text{CurrentRequiredSources}(p,t) \wedge \text{ValidMandate}(d,t) \wedge \text{PolicyAllows}(p) \wedge \text{BudgetReserved}(p,t).$

$\text{MayApply}(n,t) = \text{AdmitStillApplicable}(n,t) \wedge \text{OwnerAuthorizes}(n,t) \wedge \text{DependenciesReceived}(n).$

These are specification predicates. Their components are enforced across different boundaries, not by an atomic global check. Correct composition requires each owner to validate the relevant epoch, scope, mandate, and fence when admitting its own effect. A cached positive decision cannot eliminate that requirement.

3.3 Failure and trust model

The model allows controller crashes, retries, delayed or lost responses, concurrent proposals, stale observations, and temporary disconnection from required services. An effect may have completed even when its caller received no response. A requester may submit malformed or substituted plans or attempt to reuse expired decisions.

Conditional properties require the following deployment assumptions:

1. Owners authenticate requests and enforce their own scope, epoch, authorization, and resource checks. Protected effects cannot bypass these boundaries.
2. A stable operation identity refers to the same immutable operation. Owners either deduplicate and reconcile that identity reliably or report uncertainty without authorizing blind replay.
3. Durable stores provide the required conditional-write semantics. History rollback is detected using a separately trusted witness where that deployment profile requires one.
4. Digest bindings use canonical serialization and collision-resistant hashing. A digest alone provides neither authentication nor evidence truthfulness.
5. Time and expiry validation follow a documented deployment contract. Progress requires eventual availability of the owners needed for the transition; safety may require indefinite blocking.

Compromised authority owners, stolen signing keys, arbitrary provider dishonesty, unmediated tool use, and undetectable storage rollback are outside these conditional guarantees. The protocol does not establish semantic correctness of an LLM output or prove that a declared capability is useful.

4. Transition protocol

Figure 1 separates persistent collaboration, organizational transition, and effect authority. The arrows denote data and control dependencies, not a global transaction.

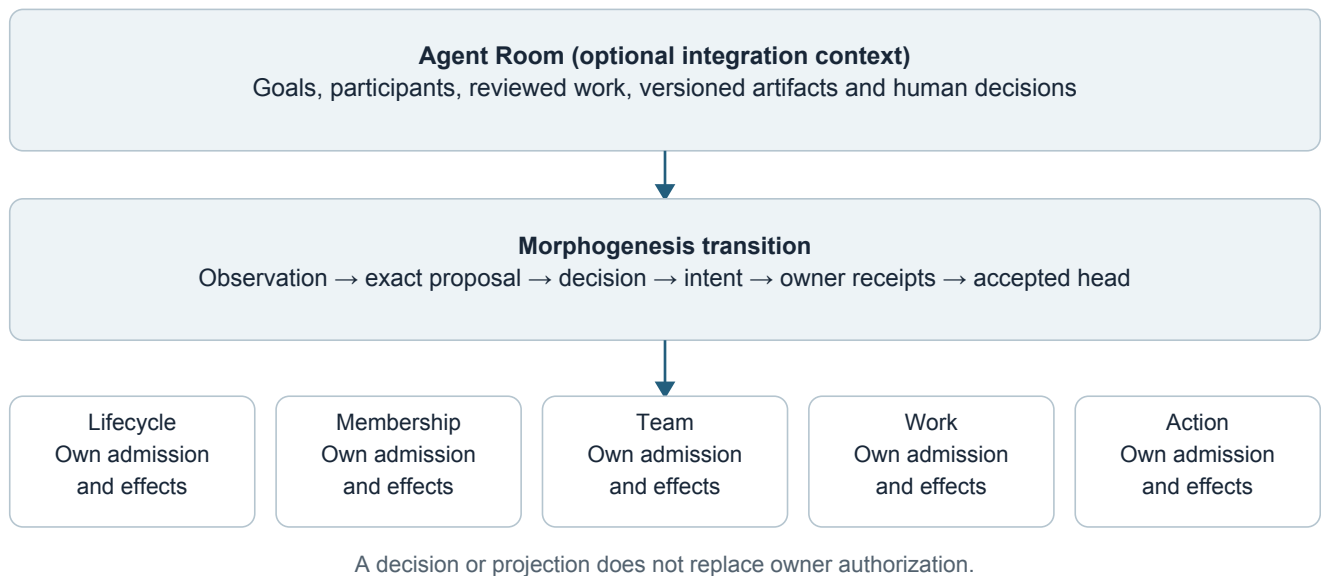


Figure 1. Agent Room preserves collaboration records. Morphogenesis binds the proposed change and its evidence. Lifecycle, Membership, Team, Work and Action retain their own admission boundaries. An artifact projection grants no operational authority.

4.1 Observe, propose, and authorize

The controller validates required source heads and records a bounded need. It selects an admissible target and compiles the operator plan. In the catalog lifecycle profile, the process definition comes from a closed catalog; a proposal cannot install arbitrary workflow code. Advanced operator compilation still uses a closed vocabulary and existing owners.

The proposal binds its source epoch and exact decision subject. A workflow gate may select an approved branch, but branch selection is not sufficient authorization. The controller resolves and validates the decision before preparing effects. The advanced governed entry point additionally binds the compiled plan to a separately issued execution authorization and fence.

Budget reservation accounts for concurrent proposals before resource materialization. It is distinct from observed workflow usage and from budgets enforced by downstream owners. Reservation failure rejects or delays the transition; a reserved global envelope cannot widen an individual work or action grant.

4.2 Prepare, enact, and reconcile

Before each external operation, the controller persists immutable intent and its stable identity. On restart, it asks the owner to reconcile any prepared operation whose result is unknown. A confirmed result can supply a validated receipt. A definitely absent effect may be attempted only when the owner contract makes that determination safe against delayed execution. An indeterminate result blocks advancement.

The protocol therefore does not equate a timeout with a failed effect. Nor does an application-side journal itself ensure exactly-once provider behavior. Those properties depend on the effect owner's implementation and the external system it controls.

An abstract execution rule is:

```

for each ready effect in the approved plan:
  persist immutable intent under its stable operation identity
  validate current owner admission requirements
  resolve any previously prepared attempt with the owner
  if its outcome remains indeterminate: stop advancement
  if safely unapplied: request the effect under the same identity
  validate and persist the owner's result receipt
  attempt the morphology-head update against the bound predecessor
  
```

This is explanatory pseudocode. It does not replace the distinct V1 and V2 runtime state machines, nor imply that an owner call and a journal write share a transaction.

4.3 Commit and recovery boundaries

The morphology store accepts a successor using revision-and-digest compare-and-swap. A competing update invalidates the expected predecessor; it does not authorize silently rebasing an old decision onto the new state.

Owner effects and morphology commitment are separate events. In the inspected V1 implementation, successor Team activation precedes morphology commitment. Consequently, a controller may recover with an activated Team and an unadvanced morphology head. This intermediate condition must be reconciled from owner receipts. A single winning morphology head is not sufficient evidence that all losing proposals had no material effects.

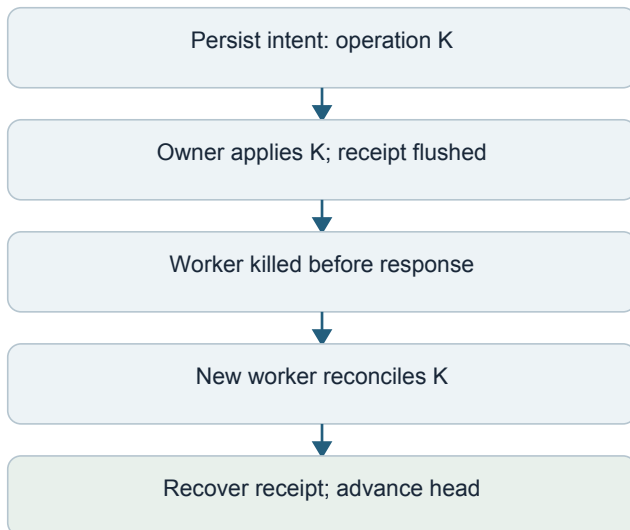
Declared reversible effects may be compensated before commitment. The advanced profile journals compensation separately and processes applied steps in reverse order. Irreversible effects and unresolved outcomes require explicit handling; they cannot be erased by restoring an old projection. After commitment, recovery is represented as a successor transition rather than an atomic rollback of history.

4.4 A losing proposal can leave an effect

Consider two proposals A and B bound to morphology epoch e . Each obtains an owner receipt for Team activation before either updates the morphology head. A commits first. B's update is rejected as stale, but B's owner receipt still exists. The pilot in Section 7 reproduces this boundary with the real head runtime and synthetic Team effects.

The correct observable is therefore a pair: accepted successors and material effects from losing proposals. Reporting only one accepted successor hides the second quantity. A deployment must specify whether pre-commit Team activation can admit useful work, which owner fences constrain it, and how losing effects are reconciled or compensated. This paper does not assert that V1 automatically compensates every such effect. The probe is a counterexample to a stronger interpretation of CAS, not a demonstrated bypass of all integrated admission controls.

A. Lost acknowledgement



B. Competing successors

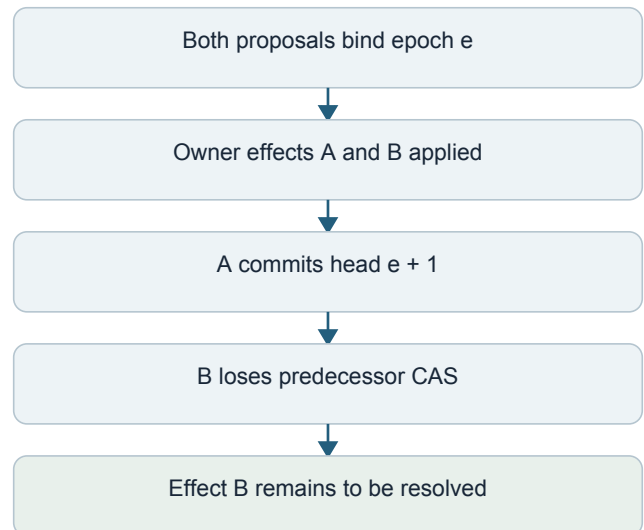


Figure 2. Left: durable intent and an owner receipt survive worker termination; reconciliation recovers the same operation. Right: two owner effects may precede a single accepted head. Compensation or successor recovery is a separate obligation.

4.5 Work continuity and retirement

Detachment and retirement require a continuity checkpoint and fencing of work/action authority before removing or terminating the resource. The checkpoint identifies useful work and its provenance; successor work allocation remains owned by the work subsystem. Material availability, membership, team participation, and permission to act are distinct states.

For the specialist example, the transition records why the agent was added, which profile and decision authorized the plan, which work it received, and which receipts established its later removal. If a retirement acknowledgement is lost, reconciliation uses the original operation identity. Causal continuity means these dependencies remain reconstructible; it does not mean that every mission succeeds or that every generated artifact remains semantically useful.

4.6 Three meanings of continuity

For an accepted transition, define evidence continuity as the ability to resolve a typed dependency path from predecessor and observation through proposal, decision, intent, owner receipts, and successor. Every edge must have matching scope and immutable subject bindings. Hash agreement alone does not establish source authenticity or retention.

Work continuity concerns artifacts: for a nonempty set of predecessor artifacts required by successor tasks, measure the fraction still addressable at the required version with valid dependencies. An empty required set is reported as not applicable, not perfect continuity. Semantic validity needs a task oracle; existence of an artifact is insufficient.

Mission continuity concerns purpose and constraints across multiple transitions. It is outside the core evaluation. Here, causal continuity denotes execution provenance and dependencies, not a causal estimate of performance improvement.

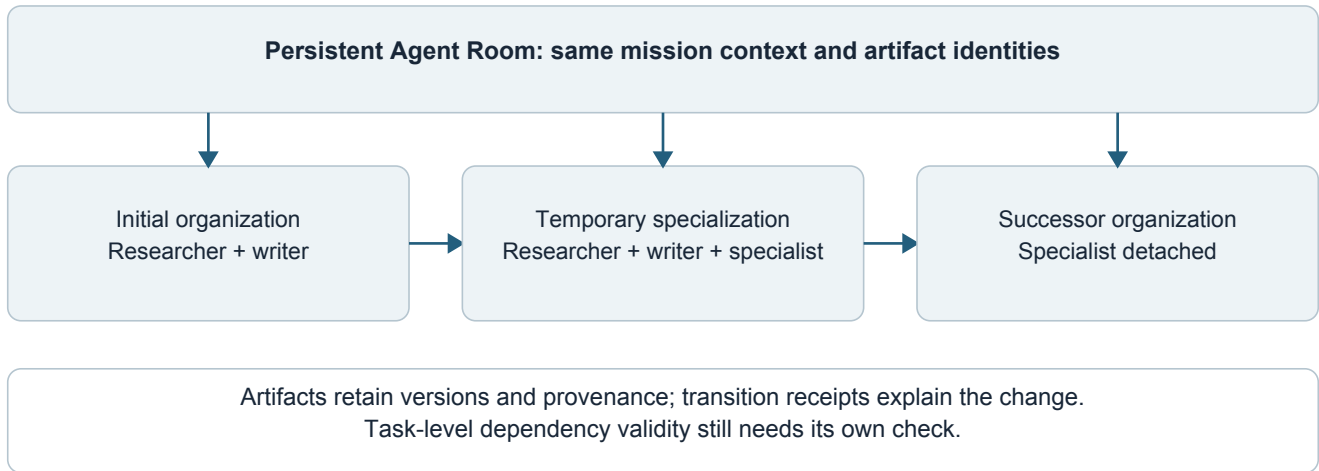


Figure 3. The Room and artifact identities can persist while participants change. An accepted receipt explains a transition; artifact dependency checks determine whether prior work remains useful. Neither property alone proves mission fidelity.

5. Conditional properties and proof obligations

The following arguments apply to the abstract model under Section 3's assumptions. They are proof sketches and implementation obligations, not machine-checked theorems about the entire repository.

P1 — Owner-bounded authority. Every admitted protected effect has a current grant from its owner. Assuming complete mediation and correct owner validation, induction over effect admissions establishes this property: coordination records create no alternative admission path. An implementation that invokes an unprotected provider bypasses the premise. This property does not imply that an authorized policy is ethically appropriate or that authority can never be expanded through a legitimate owner decision.

P2 — One accepted successor per predecessor revision. If all writers use the same authoritative, linearizable conditional store and the expected revision/digest pair, at most one distinct successor can consume that pair. The first successful write changes the comparison value; a competing write cannot succeed against it. This is a local head property. Restored stores, disconnected authority replicas, and material side effects require additional mechanisms and are not covered by CAS alone.

P3 — No blind replay of unresolved effects. If every effect is preceded by durable intent, recovery inspects that intent, and indeterminate owner results block execution, controller recovery cannot intentionally resubmit the

unresolved operation as a fresh operation. Preventing duplicate material effects additionally requires correct owner deduplication and reconciliation. A delayed original request racing with an incorrect “absent” answer is a counterexample when those assumptions fail.

P4 — Terminal removal follows fencing. If the terminal boundary requires validated checkpoint and fence receipts and the owners enforce those fences at action admission, resource removal follows the declared continuity and revocation steps. Already admitted in-flight actions require the owner's draining contract; fence issuance does not retroactively undo them.

P5 — Accepted lineage is evidence-linked. If the head accepts only validated receipt bindings and predecessor references, each accepted successor has a reconstructible dependency path to its proposal and decision. Hash linkage detects some inconsistencies but does not prove that an external receipt is truthful, complete, or durably retrievable. Evidence retention is a separate operational obligation.

Property	Runtime obligation	External premise and counterexample
P1: authority	Route protected effects through their owners	Correct use-time grants; a bypassing tool call defeats mediation
P2: successor	Compare the expected predecessor in the head store	Linearizable store; effects from a losing proposal remain possible
P3: recovery	Retain intent and operation identity; stop on uncertainty	Reliable owner reconciliation; a false absent response permits duplication
P4: removal	Require checkpoint and fence before terminal removal	Owners enforce fences and draining; already-admitted work can remain active
P5: evidence	Validate predecessor, decision and receipt bindings	Retain authentic evidence; a valid digest cannot recover a deleted object

The table allocates obligations rather than assuming that composition automatically establishes them. In particular, a synthetic owner rejection demonstrates that a denial propagates through the coordinator, not that an independently deployed owner correctly implements authorization.

Liveness is intentionally weaker than safety. An unavailable owner, expired decision, lost budget reservation, or indeterminate effect can block progress. A meaningful evaluation must count those blocked missions and their recovery cost instead of treating all refusals as successful adaptation.

6. Reference implementation

AgentPlat implements Morphogenesis as an opt-in subpath of AgentPlat Collective Runtime. Domain contracts represent observations, proposals, decisions, execution records, and receipts. Governed Durable Workflows owns process execution. The Collective Host PostgreSQL adapter owns durable morphology storage. Agent Rooms and Agent Mesh expose projections without acquiring execution authority. The package architecture preserves existing lifecycle, Membership, Team, Work, Trust, and Action boundaries. See the architecture and ADR 0046.

The paper's core scope corresponds to the V1 specification, with compiled operators and compensation from V2. Selected source inspection confirms explicit plan/decision matching and authorization expiry checks in the governed V2 entry point, prepared-operation reconciliation in the V1 execution runtime, and an indeterminate compensation state in V2. These observations establish source structure, not deployment-wide enforcement.

The implementation uses content-minimized control records: identifiers, enums, counters, and digests rather than raw prompts, model outputs, or private keys. Such records can still expose metadata and correlations. Content minimization is not a general privacy proof.

6.1 Agent Room integration

An Agent Room supplies persistent goals, participants, versioned artifacts, approvals and provenance [12]. In AgentPlat, `projectMorphogenesisDiffToRoomArtifactV1` and `projectMorphogenesisReceiptToRoomArtifactV1` expose transition proposals and outcomes as Room artifacts. `MorphogenesisRoomParticipationPortV1` adds a participant with authority level zero and no permissions; it does not mint Mesh membership or Work authority. The authorized-person decision adapter binds a Room approval to an exact candidate and mandate. Approval of an unrelated artifact cannot stand in for that decision.

The implementation's `roomId` may be null. Rooms are therefore a concrete integration context, not a requirement of every protocol deployment. In the running example, the Room retains the proposal's work while a specialist joins and later leaves the active organization. The local pilot checks artifact addressability across a transition; semantic dependency validity, participant lifecycle synchronization and durable Room recovery require additional integration evaluation.

6.2 Version and capability boundaries

The source snapshot for the v0.2 inspection and pilot is `e978544915a329387e13883fa352191f6993dcc7`, together with the separately hashed pilot harness. The v0.1 draft inspected `12fd870bb858476fc0f221d13b6ee1f6b1ab3e39`. The pilot environment manifest binds the executed compiled JavaScript and lockfile; historical results retain their original bindings. AgentPlat Agent Morphogenesis remains outside the frozen Collective Capability Baseline V1 denominator.

Capability	Available source surface	Evidence in this paper
V1 lifecycle and recovery	Runtime, owner ports, head and execution stores	Source mapping and selected local boundary executions
V2 operators and compensation	Governed plan admission and compensation journal	Source mapping; existing tests, no new V2 campaign
V3-V4 strategy adaptation and exchange	Opt-in source capabilities	Outside the pilot
V5-V6 synthesis and Agent Genesis	Opt-in source capabilities	Outside the pilot
V7-V8 organizational and constitutional evolution	Opt-in source capabilities	Outside the pilot; longitudinal claims remain open

7. Reproducible boundary pilot and historical evidence

7.1 Method and artifact

We executed eleven prescribed local cases using the existing V1 runtime, an instrumented synthetic Team owner, and the real morphology-head implementation. The recovery cases run in separate Node.js processes. A single-writer JSONL test adapter flushes writes and replays them through the existing reference store validators; it is not a supported production adapter or a test of PostgreSQL/Temporal. The owner independently records each materialized synthetic effect.

The lost-response schedule sends SIGKILL to the worker after the owner receipt is flushed and before it returns. The next worker reopens the files. Three conditions share that owner: the V1 runtime; a fresh-identity retry ablation; and a small durable-pattern controller that records intent and reconciles a stable operation identity. The comparator exercises only Team-effect recovery. It is not a complete saga implementation or an end-to-end framework comparison. Counts are compared; performance is not.

The design was written before the harness was first executed. There is one execution per prescribed schedule/condition, not a random sample. Two preliminary attempts failed on a harness mistake when reading Room artifact content from metadata instead of its version. Those failures are retained separately and are not passing evidence. Another pilot attempt succeeded before the final bundle and verifier checks. No model calls or

paid services were used.

The pilot protocol, runner, and verifier are included in the repository. The pilot/ bundle includes worker outcomes, process identities, owner calls, effect receipts, execution/head journals, environment bindings and a SHA-256 manifest. A separate verification pass recounts effects and validates record bindings. This is reproducible local verification, not an independent laboratory replication.

7.2 Observations

Boundary and condition	Observed outcome	Interpretation
Nominal: all three conditions	One Team effect each	Common nominal behavior at this boundary
Lost response: V1 runtime	One effect after worker replacement	Stable-ID reconciliation recovers the receipt
Lost response: durable-pattern comparator	One effect after worker replacement	Recovery benefit is shared with an established durable pattern
Lost response: fresh-identity ablation	Two effects on the same semantic Team target	A new identity escapes per-operation deduplication
Owner unavailable during recovery	Blocked before head advancement; resumed after restoration	Unknown outcome does not become fresh execution
Current owner denies admission	No Team effect; owner rejection retained	Earlier approval does not override the synthetic owner
Two effects followed by competing head updates	One accepted successor, one losing-proposal effect	Head consistency is weaker than transition atomicity
Detachment attempted before fence	Rejected; normal completion after fencing	V1 enforces the tested phase prerequisite
Room artifact/receipt projection	Earlier work version retained; receipt retrievable	Addressability in the in-memory Room repository

The competing-head probe applies two synthetic owner effects and then submits competing commits to the real head runtime. It does not execute two complete independently approved transitions and does not test automatic cleanup. The owner-denial fixture returns a fixed rejection after organizational approval; it does not exercise a real mandate-expiry race. Similarly, the Room case does not evaluate semantic artifact quality. The pilot intentionally reports these narrower boundaries instead of turning them into broader system guarantees.

7.3 What the pilot adds

The lost-response comparison does not establish unique recovery superiority: the durable-pattern comparator also preserves a single effect. The contribution under examination is the organizational composition and its contracts. The losing-effect trace provides a concrete, retained example of an integration obligation that a successful head CAS alone cannot discharge. Source-linked code and raw effects make both observations inspectable.

7.4 Historical evidence remains separate

The Beta 1 report, source 763b0429b4db4d931db1ddd605f6ea89ff271767, lists 18 mixed diagnostic and source-conformance scenarios. The September 9 account reports 76 runtime tests, 1,000 deterministic in-memory runs, service-backed local checks, and a subsequent supervisor snapshot 51529cfa8d4b9b482af366cbbd75e9ddcca3bf77. Its 174 ms resume time is one controlled observation. These records are contextual evidence, not additional samples of the v0.2 pilot. See the historical account.

The separate eight-scenario V1-V8 distributed staging registration still has execution disabled. It provides an experiment design, not a result. Neither the historical records nor the new pilot establish production reliability, organizational improvement, general exactly-once behavior or independent failure-domain tolerance.

8. Confirmatory mission evaluation - not executed

8.1 Research questions and conditions

RQ1: Does the composed protocol prevent the declared violations under controlled failure schedules, and which mechanisms are responsible? **RQ2:** What latency, resource cost, and blocked-progress overhead does governance introduce? **RQ3:** Under changing task requirements, when does organizational adaptation improve mission outcomes at a fixed resource budget?

Use four conditions: a fixed organization; adaptive minimal coordination; adaptation implemented over a competent durable workflow/saga design; and governed Morphogenesis. Match proposal policy across adaptive conditions, owner-side protections, tasks, tools, model versions and resource ceilings. Account for all reservation and governance costs. The complete durable baseline must include its available identity, retry and authorization controls; it is not the small recovery comparator used in Section 7.

Include a fixed organization with sufficient specialists available from the start, charging for their reserved resources, so it is not made incapable by construction. Evaluate stable demand, useful specialization, misleading adaptation signals and shrinking demand. Report capability availability explicitly: it cannot simultaneously be identical and absent only from the fixed baseline. The minimal-control condition is an ablation against instrumented test effects, not a production recommendation.

For RQ1, add one-mechanism-at-a-time ablations: stale-decision binding checks, operation reconciliation, morphology-head CAS, and fencing prerequisites. Preserve other controls. A null ablation effect may reveal a masked fault, redundant protection, or a weak workload; it must not automatically be described as proof that the mechanism is unnecessary.

8.2 Workloads and fault injection

The first workload is a deterministic mission graph with independently scored artifacts and explicit capability requirements. During execution, introduce a missing skill, agent unavailability, and a later reduction in demand. This tests addition, work continuity, and removal without relying on subjective model judgments. A later LLM-backed workload can use the same transition envelope, but requires a separate model and budget registration.

Scenario	Injection point	Independent observable
Capability change	New task requires unavailable skill	Completion, adaptation delay, incremental cost
Lost effect response	Owner commits before acknowledgement	Material-effect count by operation identity
Competing successors	Two plans share a predecessor	Accepted heads and effects from losing plans
Stale approval	Owner epoch changes after decision	Rejected or admitted protected effects
Removal under load	Agent has pending or in-flight work	Fence enforcement, drain result, recoverable artifacts
Controller restart	Crash before/after selected journal boundaries	Recovery time, unresolved operations, duplicates
Owner unavailability	Reconciliation endpoint is unreachable	Blocked duration and premature advancement

Instrument effects at the owning sink, not only in controller logs. Record authorization violations as admissions that lack a valid owner grant at the relevant admission event. Count duplicate effects against declared operation semantics. Preserve receipts for losing proposals and compensated operations so that head correctness does not conceal external damage.

8.3 Measurements and analysis

Primary correctness outcomes are unauthorized admissions, duplicate material effects, distinct accepted successors for the same predecessor, terminal removal without required evidence, and incomplete terminal receipt chains. Any observed prohibited outcome falsifies the corresponding bounded conformance claim. Zero observed violations does not prove their general absence.

For utility, measure mission success against an independent task oracle, completed useful work, and total resource cost. Define continuity as the fraction of previously accepted work artifacts that remain addressable and valid for the successor's task dependencies; this proposed metric is distinct from historical receipt fields named "continuity." Report recovery latency, blocked mission fraction, reconfiguration count, tokens, and provider cost separately. Retries and governance overhead count against the same total budget.

An initial engineering pilot should assess harness correctness and runtime variance. After that pilot, freeze task instances, fault schedules, sample sizes, exclusion rules, and analysis in a new registration before confirmatory runs. A provisional design uses 30 independently seeded mission instances per workload/fault cell, paired across conditions; this is a planning value, not a power calculation. Repeated observations within one mission are not independent samples.

Report paired outcome differences with uncertainty intervals, per-scenario raw counts, and median and tail latency when enough observations exist. Include timeouts, blocked transitions, setup failures, and budget exhaustion. Predeclare the timeout horizon; treat unresolved recovery times as censored rather than silently dropping them. A descriptive zero-event upper bound is meaningful only under explicitly justified sampling assumptions, not across correlated retries or deterministic replays.

The fixed organization may perform better when no adaptation is needed, and governance may consume enough resources to reduce task success. Those outcomes would delimit the method's usefulness. RQ1 can succeed while RQ3 fails; the paper must not substitute conformance for utility.

9. Limitations and future work

The literature comparison is selective and does not prove priority over all organizational middleware, runtime verification or distributed reconfiguration. The abstract predicates also need a precise refinement mapping to runtime transitions and negative tests covering cross-owner interleavings. There is no end-to-end formal proof in this version.

The trusted computing base is substantial: identity issuers, policy configuration, effect owners, storage semantics, expiry handling, and deployment wiring. A correct library can be composed incorrectly. Resource reservation also cannot bound activity that bypasses the registered resource authorities. Local multi-process tests do not model independent physical failure domains or long-lived production traffic.

Causal evidence does not establish semantic mission fidelity. A formally authorized organizational change can be unhelpful, expensive, or based on mistaken capability evidence. Evaluating those outcomes requires task-level ground truth and matched baselines. Human or quorum decisions introduce additional costs and failure modes, including correlated reviewers and inadequate mandates.

Later AgentPlat versions explore strategy adaptation, synthesized profiles, agent genesis, whole-collective evolution, and constitutional continuity. These extensions raise a longitudinal question: can individually admissible changes accumulate into mission drift or excessive authority concentration? That question motivates future work but is not answered by the V1/V2 evidence here. Finite model exploration, when used, must report its bounds and incomplete outcomes rather than claiming unrestricted proof.

10. Conclusion

Governed agent morphogenesis treats organizational change as a coordinated transition among independently authoritative subsystems. Its central separation is between proposing a new organization, authorizing a bounded plan, and admitting each material effect. Durable intent, owner receipts, conditional successor selection, and explicit recovery make that separation inspectable across failures. AgentPlat supplies a reference implementation and a reproducible local boundary pilot. The pilot shows shared stable-ID recovery behavior and exposes the distinction between one accepted successor and residual losing effects. A persistent Agent Room can retain work and transition records across participant changes. Confirmatory mission experiments remain necessary to establish when this composition is useful and what cost its guarantees impose.

References

1. AgentPlat contributors. *AgentPlat*. Source repository; cite the exact experiment commit. [Repository](#). The source inspected for this draft and historical experiment commits are distinguished in Sections 6–7.
2. Virginia Dignum and Frank Dignum. 2012. *A Logic of Agent Organizations*. *Logic Journal of the IGPL* 20(1), 283–316. DOI; [author manuscript deposited in 2018](#).
3. Moise project. *The Moise Organisation Oriented Programming Framework*. Project documentation, accessed 2026-09-13. [Project source](#).
4. Weize Chen et al. 2023. *AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors*. arXiv:2308.10848. [Preprint record](#).
5. Siyuan Lu, Jiaqi Shao, Bing Luo, and Tao Lin. 2024. *MorphAgent: Empowering Agents through Self-Evolving Profiles and Decentralized Collaboration*. arXiv:2410.15048v2, revised 2025-09-03. [Version consulted](#).
6. Guibin Zhang et al. 2024. *G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks*. arXiv:2410.11782. [Preprint record](#).
7. David Garlan, Shang-Wen Cheng, An-Cheng Huang, Bradley Schmerl, and Peter Steenkiste. 2004. *Rainbow: Architecture-Based Self Adaptation with Reusable Infrastructure*. *IEEE Computer* 37(10). [Author publication page](#).
8. Hector Garcia-Molina and Kenneth Salem. 1987. *Sagas*. Princeton University technical report TR-070-87. [Institutional record](#).
9. Leslie Lamport. 1978. *Time, Clocks, and the Ordering of Events in a Distributed System*. *Communications of the ACM* 21(7), 558–565. [Author publication page](#).
10. Alexandru Sorici, Gauthier Picard, Olivier Boissier, Andrea Santi, and Jomi F. Hübner. 2012. *Multi-Agent Oriented Reorganisation within the JaCaMo Infrastructure*. ITMAS author manuscript, Sections 3-4. [Full text](#).
11. Mateusz Guzek, Grégoire Danoy, and Pascal Bouvry. 2013. *ParaMoise: Increasing Capabilities of Parallel Execution and Reorganization in an Organizational Model*. *AAMAS*, 1029-1036. [Proceedings paper](#).
12. Douglas Javier Rodriguez. 2026. *Agent Rooms: A Conceptual Framework for Persistent Human-Governed Multi-Agent Collaboration*. Version 2. [Zenodo record and paper](#), DOI: 10.5281/zenodo.20564834.

The source register accompanying this paper records the depth of inspection and separates literature-supported statements, implementation observations and experimental findings. Published historical artifacts retain their original source commits.